Assignment 1
Large Language Models
Building an Urdu BPE Tokenizer from Scratch

Rohail Najmi
481893

Hasaan Hameed
454987

M. Ali Mubashir
461155

BSCS-13D

Data files (train.txt, val.txt, test.txt, train.bin, val.bin):
https://drive.google.com/drive/folders/1yp_-1ByzBAsVilvB4an8etXKOQl0CGNU?usp=sharing

1. What we built

We collected about 1.30 GB of clean Urdu text from four sources: VOA Urdu (news, which we scraped ourselves), Urdu Wikipedia (encyclopedia), Urdu Wikisource (poetry and literature) and MADLAD-400 (web text). Full details are in `DATA_CARD.md`. We then wrote a byte-level BPE tokenizer in plain Python and trained it on `train.txt` to a vocabulary of exactly 50,000 tokens.

The tokenizer starts from the 256 possible bytes. It then repeatedly finds the two pieces that appear next to each other most often and joins them into a new token. After 49,743 joins (called *merges*), it has a vocabulary made of letters, word parts and whole words.

Vocabulary	256 bytes (IDs 0–255), the end-of-document marker `<|endoftext|>` (ID 256), and 49,743 merges (IDs 257–49,999)
Training time	307 seconds on the full `train.txt` (1,276 MB), on an Intel Core Ultra 7 155U laptop (12 cores, 15.4 GB RAM)
Encoded corpus	`train.bin` has 206.3 million tokens and `val.bin` has 2.0 million
Correctness check	The slow reference trainer and the fast trainer learned exactly the same first 5,000 merges on a 1 MB sample. The fast one took 2 seconds, the slow one 26 minutes.

How text is split into words first

Before training, the text is cut into words so that a merge never joins the end of one word with the start of the next. We use this pattern:

```
?[\p{L}\p{M}\u200c]+| ?\p{N}{1,3}| ?[^\s\p{L}\p{M}\p{N}]+|\s+(?!\S)|\s+
```

- Letters, diacritics and ZWNJ stay together. Zer, zabar, pesh and shadda count as part of the word. A word like اُردُو stays whole, whereas GPT-2's pattern breaks it at every diacritic.
- The space before a word belongs to that word, the same as in GPT-2.
- Numbers are cut into groups of up to three digits, so the tokenizer doesn't waste merges on long numbers.
- Every character fits one of the rules, so nothing is ever lost. Decoding always gives back the exact original text.

Normalization is a separate step, done on the corpus before training. It changes Arabic letters to their Urdu forms, turns all digits into 0–9, and keeps ZWNJ only between letters.

2. Q1: What did the merges learn?

The first 50 merges

Most Urdu letters take 2 bytes. A dash (—) means the token is only part of a letter, so it can't be shown as text on its own.

#	Token	Type	#	Token	Type
1	—	space + half of a letter	26	۔	full stop (۔)
2	ی	letter	27	ب	letter
3	ا	letter	28	گ	letter
4	ک	letter	29	کے	space + word ("of")
5	—	space + half of a letter	30	ور	letter pair
6	ر	letter	31	ت	space + letter
7	و	letter	32	کی	space + word ("of")
8	ہ	letter	33	پ	space + letter
9	ے	letter	34	ن	space + letter
10	ک	space + letter	35	ار	letter pair
11	ن	letter	36	ان	letter pair
12	ا	space + letter	37	ج	space + letter
13	ل	letter	38	میں	space + word ("in")
14	ت	letter	39	ٹ	letter
15	م	letter	40	د	space + letter
16	م	space + letter	41	ع	letter
17	س	letter	42	و	space + letter
18	ں	letter	43	ق	letter
19	—	letter + half of the next letter	44	ہے	space + word ("is")
20	ہ	space + letter	45	ف	letter
21	یں	letter pair	46	اس	space + word ("this")
22	س	space + letter	47	اور	space + word ("and")
23	ب	space + letter	48	ل	space + letter
24	د	letter	49	ائ	letter pair
25	ھ	letter	50	ری	letter pair

Every 5,000th merge

Merge	Token	What it is	Merge	Token	What it is
5,000	مسکر	start of مسکرانا (to smile)	30,000	تیندوے	leopards
10,000	فرینڈ	"friend", an English loanword	35,000	372	a three-digit number
15,000	ولاد	part of ولادت (birth) and اولاد (children)	40,000	وہیں	"right there"
20,000	محافظوں	guards	45,000	سمبل	"symbol", an English loanword
25,000	ged	English word ending (as in "changed")	49,743	چانڈیو	the surname Chandio

What these tables show

- Rebuilding letters. Every Urdu letter is stored as 2 bytes, so the first job of the tokenizer is to glue those bytes back into letters. 22 of the first 50 merges do exactly this. The most common letters come first (ک, ا, ی). Rarer letters come much later (ڑ at merge 1,177), as do diacritics (zer at 164, pesh at 225). Across the whole vocabulary, 325 merges rebuild a two-byte character.

- Tokens that are not valid text on their own. Only 3 of the first 50 merges (1, 5 and 19), and 208 of all 49,743 (0.4%), are just part of a letter. Merge 1 is a space followed by the first half of a letter. Almost every Urdu word starts with a space and one of a few common first bytes, so this pair was the most frequent of all. These half-letters are used as building blocks: merge 12 is built on top of merge 1. Most of the later ones are pieces of rare symbols such as curly quotes, or Hindi and Bengali script that appeared in Wikipedia.
- The first whole word appears at merge 29: کے ("of"), which appears 61,451 times in `test.txt`. It is quickly followed by ہے, میں, کی and اور, the small words that connect almost every Urdu sentence. By merge 5,000 the tokenizer is learning normal content words, and by the end it is learning rare names like Chandio.

The 20 longest tokens

Merge	Token	Letters	Merge	Token	Letters
40,417	hadithtransmitters	19	49,101	Nottinghamshire	16
36,859	shiaonlinelibrary	18	3,083	CricketArchive	15
37,216	theatreworldawards	18	4,728	snaccooperative	15
40,831	islamansiklopedisi	18	10,215	انسائیکلوپیڈیا (encyclopedia)	15
2,692	minorplanetcenter	17	18,763	Administration	15
15,494	githubusercontent	17	19,888	Transportation	15
26,115	a row of 16 Urdu full stops (.)	16	20,864	Internationals	15
31,975	*-*-*-*-*-*-*-*-	16	21,507	cryptocurrency	15
35,132	Recommendations	16	23,549	Synchronkartei	15
40,166	Gloucestershire	16	28,865	frauendatenbank	15

Surprisingly, the longest tokens are mostly English, and most of them are not even normal words. They are website names that appear in the reference lists of thousands of Wikipedia articles, such as *githubusercontent* and *minorplanetcenter*, plus decorative separator lines. Because they repeat so often, the tokenizer learned them as single tokens. The longest real Urdu tokens are borrowed words such as انسائیکلوپیڈیا (encyclopedia) and انٹرکانٹینینٹل (intercontinental). This tells us some of our merges were spent on Wikipedia's repeated reference text, which a stricter cleaning step could remove.

3. Q2: How does it compare with existing tokenizers?

We counted how many tokens each tokenizer needs per word, where a word is anything between spaces. Fewer tokens per word is better: the text takes less space, and a model can fit more of it into its context window. We compared our tokenizer with GPT-2's (`gpt2`) and a modern multilingual one (`o200k_base`). The inputs were our test set (2,880 unseen documents) and seven short passages we picked by hand. The passages are saved in `analysis/samples/`.

Input	Words	Ours	GPT-2	o200k_base
test.txt	1,701,409	**1.29**	5.77	1.71
News	68	**1.06**	6.10	1.35
Poetry (Ghalib, Iqbal)	51	**1.14**	5.61	1.57
Urdu with English words	43	1.37	3.88	**1.16**
Roman Urdu	34	2.56	2.32	**1.97**
Numbers and dates	44	**1.52**	5.27	1.86
Text with diacritics	29	**1.83**	7.48	2.83
Personal names	33	**1.48**	7.48	2.36

Tokens per word. The best result in each row is in bold.

On normal Urdu, our tokenizer is the most efficient of the three. On the test set it needs 1.29 tokens per word. `o200k_base` needs about a third more, and GPT-2 needs about 4.5 times as many. GPT-2 was trained mostly on English, so it barely knows any Urdu and breaks most Urdu words into single bytes. On our news and poetry samples

we are close to one token per word, so most words become a single token. For Assignment 2 this means our model can read several times more Urdu text in the same context window.

Outside Urdu script, the other tokenizers do better. When English words are mixed in, and especially for Roman Urdu, `o200k_base` wins. For Roman Urdu even GPT-2 beats us. This is expected: almost all of our 50,000 tokens were learned from Urdu text, while the other two were trained on huge amounts of English. Diacritics and names cost us a bit more than plain news, but we still need far fewer tokens than both other tokenizers there.

4. Q3: Three inputs our tokenizer handles badly

1. Roman Urdu: 2.64 tokens per word

Input: Mujhe Urdu bolna bohat pasand hai lekin likhna mushkil lagta hai

Result: 29 tokens for 11 words. Common words are cut into small pieces, for example *bohat* → b + oh + at, and *mushkil* → m + ush + k + il. The same sentence in Urdu script (مجھے اردو بولنا بہت پسند ہے لیکن لکھنا مشکل لگتا ہے) needs only 11 tokens, one per word.

Why: Our cleaning step kept only lines written in Urdu script, so the tokenizer almost never saw Roman Urdu during training (this gap is noted in the data card). The few English-letter tokens it has come from English names and references in Wikipedia, not from Roman Urdu. `o200k_base` handles this sentence better, at 1.82 tokens per word.

2. Urdu typed with Arabic letters and digits: 2.80 tokens per word

Input:

پاكستان ١٤ اگست ١٩٤٧ كو آزاد هوا اور يه دن هر سال منايا جاتا هے

Result: 42 tokens for 15 words. After normalization, the same sentence needs only 16. For example, پاكستان becomes 4 tokens instead of 1, and the year ١٩٤٧ becomes 8 tokens instead of 2.

Why: This sentence uses the Arabic versions of kaf, heh and yeh, and Arabic-style digits, which is common when someone types on an Arabic keyboard. They look the same as the Urdu letters but are different characters inside the computer. Our normalizer converted all of them to Urdu letters and normal digits before training, so the tokenizer never saw them and learned nothing about them. It falls back to raw bytes, and each such letter breaks the word it sits in. Fix: always run `normalize()` before encoding, as our pipeline does. The tokenizer itself must not change the text, because it has to give back exactly what it was given.

3. Emoji: 5.33 tokens per word

Input:

بہت خوب 😂😂🔥👍🏽

Result: 16 tokens. The two words take 2 tokens, and the four emoji take the other 14. Each emoji becomes 2 or 3 meaningless pieces, and the thumbs-up with a skin tone becomes 6.

Why: An emoji takes 4 bytes, and a skin-tone emoji takes 8. Our corpus is news, encyclopedia and literature, which almost never contains emoji, so the tokenizer learned only a few small byte pairs for them. Nothing is lost, and decoding still gives back the exact emoji, but chat-style Urdu full of emoji would be expensive. `o200k_base`, which was trained on web and chat text, uses 10 tokens for the same input.

All results can be reproduced with `python -m analysis.merges_report` (Q1), `python -m analysis.compare_tokenizers` (Q2) and `python -m analysis.failure_cases` (Q3).